本科毕业设计（论文）任务书

一、课题基本信息

项目	内容
课题名称	面向代码智能体的开发任务上下文追踪与代码变更溯源系统设计与实现
英文名称	Design and Implementation of a Task-Context Tracking and Code-Change Provenance System for Code Agents
学院 / 专业	待填写
学生姓名 / 学号	待填写
指导教师 / 职称	待填写
课题来源	自拟（结合代码智能体方向的前沿问题与工程实践）
课题性质	工程设计（含研究性内容）
起止时间	2026 年 11 月 — 2027 年 5 月

二、课题背景与意义

近年来，以大语言模型为决策核心的代码智能体（code agent）已从实验室走向日常开发。开发者只需用自然语言描述任务，智能体即可自主完成代码检索与理解、文件读写、命令与工具调用、测试执行以及错误排查。以 Claude Code、Cursor、Cline、OpenHands 等为代表的工具，正在把开发者的工作重心从"逐行编写代码"转向"描述任务、审阅修改、验证结果"。

但现有工具的信息组织仍以"会话"或"文件"为单位：对话历史在一个面板，代码差异在另一个视图，工具调用记录散落于终端与日志，测试结果属于另一套报告，版本历史则另存于 Git。这带来三个实际问题。其一，**缺乏任务视角**。一次开发任务往往包含多轮对话、多个文件的交叉修改与多次测试，这些记录不被聚合到同一个"任务"之下，开发者难以回答"这个任务现在做到哪一步了"。其二，**修改原因不可回溯**。面对一段被智能体改写的代码，开发者很难快速确认它由哪一句需求触发、经过了哪些工具调用、又由哪次测试验证过，代码审查与问题定位因此退化为人工翻找与回忆。其三，**中断后上下文丢失**。开发过程常被会议、并行任务打

断，重新开始时需要重读长对话与大量 diff 才能恢复状态，恢复成本高，且容易遗漏未完成的修改与待验证的假设。

已有研究为上述问题提供了部分基础，但尚未形成完整方案。在智能体能力方面，SWE-bench[1] 建立了以真实仓库 Issue 为驱动的评测基准，SWE-agent[2] 提出面向智能体的计算机接口设计，OpenHands[3] 与 CodeAct[4] 探索了统一的动作空间与可执行交互，MetaGPT[5]、ChatDev[6] 研究了多智能体协作的开发流程，其关注点主要在"任务能否完成"，而非开发过程信息的组织与回放；国内亦有工作系统梳理了检索增强生成在软件工程中的应用[17]，并探索了基于大语言模型的故障复现测试用例生成[18]，但同样未涉及开发过程记录的组织。在代码演化方面，软件仓库挖掘已有系统综述[7]，基于变更的演化分析与抽象语法树差分技术[8,9]使细粒度变更识别成为可能，CodeShovel[10] 进一步构造了函数级源代码变更历史，但这些技术通常不涉及自然语言对话这一触发来源。在溯源与解释方面，Whyline[11] 通过记录执行轨迹回答开发者关于程序行为的"为什么"问题，数据溯源的理论体系[12]与 noWorkflow[13] 等系统为过程证据的组织提供了方法基础，但均未针对"人类自然语言指令驱动的代码变更"这一场景给出模型。在中断恢复方面，已有实证研究揭示了开发者中断后的恢复策略及其对可用线索的依赖[14]。在工具现状方面，LangSmith、Langfuse 等可观测性平台[16]能够记录调用链与提示词，OpenTelemetry 也已提出生成式 AI 的语义约定[15]，但其面向调用调试与成本监控，不与代码变更、测试结果做语义级对齐；IDE 的本地历史只保存文件快照，不记录修改动因与验证依据。由此可见，**缺少以开发任务为中心、跨对话/代码/执行/测试四类异构记录的统一关联模型，以及面向开发者的可交互溯源与上下文恢复方法**，正是本课题的切入点。

本课题的意义体现在三个层面：在**工程实践**层面，系统可降低开发者的上下文维护负担，缩短任务恢复时间，提高代码审查与问题定位效率；在**方法研究**层面，课题将给出面向开发过程的"任务—对话—变更—执行—测试"关联模型与溯源方法，可为智能体开发过程的可解释性与可审计性研究提供参考；在**教学训练**层面，

课题覆盖数据建模、事件采集、程序分析、前后端开发与实验评估等完整环节，能够系统训练学生的工程与研究能力。

三、主要研究内容

本课题要求学生完成以下五个方面的工作。

（1）开发任务过程的信息模型与统一事件表示。 分析代码智能体在典型开发任务中产生的记录类型，抽象出开发任务（Task）、对话轮次（Turn）、消息（Message）、工具调用（ToolCall）、文件变更（FileChange/Hunk）、测试运行（TestRun）、测试用例结果（TestCaseResult）与任务检查点（Checkpoint）等实体及其关系；设计可扩展的事件模式与存储结构，使智能体会话日志、文件系统变更、Git 提交与测试输出等不同来源的记录能够被统一表示。要求学生给出实体关系图、事件字段定义与数据库表结构。

（2）上下文采集与信息关联方法。 实现会话记录、工具调用、文件系统变更与测试结果的采集；研究将异构记录关联到同一任务的方法，包括基于时间窗口的邻近关联、基于显式引用的关联（文件路径、符号名、行号、命令、测试用例名）以及基于代码位置映射的关联（diff 坐标到当前文件坐标的映射与行号漂移处理）；对无法自动关联的记录，设计置信度标注与人工确认机制。要求学生给出关联规则集，并在标注数据上报告关联的准确率与召回率。

（3）代码变更溯源与证据链构建。 以变更块（hunk 或代码片段）为基本粒度，为每一次代码修改生成"变更卡片"，记录其触发来源（用户请求与相关对话片段）、实施过程（相关工具调用与命令）、修改前后代码快照、验证依据（执行过的测试及其结果）以及后续演化（是否被再次修改、回退或重写）。在此基础上实现"代码 → 原因 → 证据"与"对话/测试 → 受影响的代码"的双向查询，并给出典型查询的响应时间。

（4）任务上下文追踪界面与任务续接功能。 设计并实现面向单个开发者的原型界面，包括任务中心（进行中、阻塞、已完成任务及其状态）、任务时间线（按时间组织对话、变更、执行与测试事件）、文件与代码片段视角（从文件或片段反查相关任务与证据）以及变更证据侧栏。重点实现"任务续接"：任务中断后重新打

开时，系统自动生成进展摘要，列出已完成事项、未完成事项与尚待验证的修改，帮助开发者快速恢复上下文。要求学生完成可交互界面，并支持至少一次完整的"中断—恢复"演示。

（5）大语言模型辅助的摘要与关联，以及系统评估。 探索利用大语言模型生成任务摘要、归纳变更意图、整理开发进展，并辅助识别缺失的信息关联；要求所有模型输出均绑定原始证据标识（消息、变更或测试记录），保证结论可回溯，抑制模型幻觉。在此基础上设计评估方案：选取缺陷修复、功能新增、代码重构、测试补充、依赖升级等五类典型编程任务共 10—20 个，在固定开源仓库上构造实验，从代码理解、修改原因分析、问题定位与任务衔接四个维度，与"Git 历史 + 聊天记录"的基线方式对比，报告客观指标（关联准确率与召回率、溯源查询正确率、任务恢复用时、问题定位用时）与主观指标（理解正确率、NASA-TLX 认知负荷、SUS 可用性评分）。

四、基本要求

1. **系统实现要求。** 原型系统须可本地部署、可离线运行；至少接入一种真实代码智能体的事件来源（推荐命令行智能体日志导入并结合 Git 集成），并通过适配器设计为其他来源预留扩展点；数据存储采用单机方案（如 SQLite）；界面须可实际操作，不得以静态页面或截图代替。

2. **性能要求。** 在不少于 1 万条事件记录的规模下，单任务时间线加载与溯源查询响应时间应控制在 1 秒以内。

3. **可复现要求。** 系统须提供一键启动脚本与样例数据集，保证答辩现场可在无网络环境下复现全部演示流程；实验数据须由脚本生成，不得手工填写估计值。

4. **论文要求。** 论文结构完整，包含绪论、相关工作、需求分析、系统设计、关键方法与实现、系统测试与评估、总结与展望等章节；正文不少于 1.5 万字；文字通顺、图表规范（图题在下、表题在上，编号连续）；查重率须符合学院规定。

5. **学术规范要求。** 参考文献不少于 30 篇，其中外文文献不少于 10 篇，近五年文献占较大比例；所有引用须真实可查，严禁伪造文献、抄袭或代写；引用他人成果须明确标注。

6. **工作量与独立性要求。** 学生须独立完成需求分析、方案设计、编码实现、实验评估与论文撰写；每周与指导教师至少沟通一次并提交进展记录；关键设计决策须有书面说明。

五、预期成果

1. 可运行的原型系统一套，含源代码、部署与使用说明；
2. 毕业设计（论文）一份，约 1.5—2 万字；
3. 开题报告、文献综述与外文资料翻译（按学院要求）；
4. 系统设计文档与测试、评估报告，含实验数据与复现脚本；
5. 典型编程任务的演示视频与答辩幻灯片；
6. （可选）经标注的开发过程事件样例数据集，可供后续研究复用。

六、进度安排

阶段	时间	主要工作	阶段成果
一	2026.11.02—11.15	选题确认，文献调研与竞品分析，明确需求边界	文献综述初稿、需求清单
二	2026.11.16—11.30	撰写开题报告，确定技术路线与评估方案	开题报告、开题答辩
三	2026.12.01—12.20	信息模型与事件模式设计，采集方案可行性验证	设计文档、最小闭环演示
四	2026.12.21—2027.01.10	采集模块与关联算法实现	采集与关联原型
五	2027.01.11—01.20	中期检查材料整理	中期检查报告
六	2027.01.21—02.28	溯源查询、证据链与界面开发	原型 v0.5
七	2027.03.01—03.20	任务续接功能、大模型辅助摘要与关联	原型 v0.9、实验设计
八	2027.03.21—04.10	典型任务实验、用户小样本实验与数据分	实验报告与数据

		析	
九	2027.04.11—04.30	撰写论文初稿	论文初稿
十	2027.05.01—05.15	论文修改、查重与送审	论文定稿
十一	2027.05.16—05.31	答辩准备与答辩	答辩材料

七、主要参考文献

[1] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: can language models resolve real-world GitHub issues?[C]//Proceedings of the 12th International Conference on Learning Representations (ICLR 2024). 2024. arXiv: 2310.06770.

[2] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: agent-computer interfaces enable automated software engineering[C]//Advances in Neural Information Processing Systems 37 (NeurIPS 2024). 2024. arXiv: 2405.15793.

[3] WANG X, LI B, SONG Y, et al. OpenHands: an open platform for AI software developers as generalist agents[C]//Proceedings of the 13th International Conference on Learning Representations (ICLR 2025). 2025. arXiv: 2407.16741.

[4] WANG X, CHEN Y, YUAN L, et al. Executable code actions elicit better LLM agents[C]//Proceedings of the 41st International Conference on Machine Learning (ICML 2024). 2024. arXiv: 2402.01030.

[5] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: meta programming for a multi-agent collaborative framework[C]//Proceedings of the 12th International Conference on Learning Representations (ICLR 2024). 2024. arXiv: 2308.00352.

[6] QIAN C, LIU W, LIU H, et al. ChatDev: communicative agents for software development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. 2024: 15174-15186. DOI: 10.18653/v1/2024.acl-long.810.

[7] KAGDI H, COLLARD M L, MALETIC J I. A survey and taxonomy of approaches for mining software repositories in the context of software evolution[J]. Journal of Software Maintenance and Evolution: Research and Practice, 2007, 19(2): 77-131. DOI: 10.1002/smr.344.

[8] ROBBES R, LANZA M. A change-based approach to software evolution[J]. Electronic Notes in Theoretical Computer Science, 2007, 166: 93-109. DOI: 10.1016/j.entcs.2006.06.015.

[9] FALLERI J R, MORANDAT F, BLANC X, et al. Fine-grained and accurate source code differencing[C]//Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE 2014). 2014: 313-324. DOI: 10.1145/2642937.2642982.

[10] GRUND F, CHOWDHURY S A, BRADLEY N C, et al. CodeShovel: constructing method-level source code histories[C]//Proceedings of the 43rd International Conference on Software Engineering (ICSE 2021). 2021: 1510-1522. DOI: 10.1109/ICSE43902.2021.00135.

[11] KO A J, MYERS B A. Designing the Whyline: a debugging interface for asking questions about program behavior[C]//Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2004). 2004: 151-158. DOI: 10.1145/985692.985712.

[12] CHENEY J, CHITICARIU L, TAN W C. Provenance in databases: why, how, and where[J]. Foundations and Trends in Databases, 2009, 1(4): 379-474. DOI: 10.1561/1900000006.

[13] PIMENTEL J F, MURTA L, BRAGANHOLO V, et al. noWorkflow: a tool for collecting, analyzing, and managing provenance from Python scripts[J]. Proceedings of the VLDB Endowment, 2017, 10(12): 1841-1844. DOI: 10.14778/3137765.3137789.

[14] PARNIN C, RUGABER S. Resumption strategies for interrupted programming tasks[J]. Software Quality Journal, 2011, 19(1): 5-34. DOI: 10.1007/s11219-010-9104-9.

[15] OPENTELEMETRY. Generative AI semantic conventions[EB/OL]. (2026-10-27). https://opentelemetry.io/docs/specs/semconv/gen-ai/.

[16] LANGFUSE. Observability overview[EB/OL]. (2026-10-27). https://langfuse.com/docs/observability/overview.

[17] 张犬俊, 谢杨, 房春荣, 等. 检索增强生成在软件工程中的应用综述[J]. 软件学报, 2026, 37(3): 1316-1339. DOI: 10.13328/j.cnki.jos.007567.

[18] 汪莹, 字千成, 彭鑫, 等. 基于大语言模型的故障复现测试用例生成方法[J]. 软件学报, 2026, 37(4): 1690-1714. DOI: 10.13328/j.cnki.jos.007474.

八、审核意见

栏目	内容	签字 / 日期
指导教师意见		
教研室（系）意见		
学院意见		

学生（签名）：_____　　指导教师（签名）：_____

日期：_____年____月____日